Unit 4 Progress Check: MCQ Part B

1. In the following code segment, a valid Scanner object named scan is created to read from the text file input.txt.

```
File readFile = new File("input.txt");
Scanner scan = new Scanner(readFile);

int firstIn = scan.nextInt();
int secondIn = scan.nextInt();
int thirdIn = scan.nextInt();
```

Which of the following could be the contents of input.txt so that the variables firstIn, secondIn, and thirdIn are each assigned a valid value by the code segment?

(A)

123

(B) ✓

1

2

3

(C)

1, 2, 3

(D)

firstIn: 1 secondIn: 2 thirdIn: 3

Answer B

Correct. Despite being on separate lines, scan.nextInt() will read a separate int for each of the three values in the file.

Unit 4 Progress Check: MCQ Part B

2. In the following code segment, a valid `Scanner` object named `fileInput` is created to read from the text file `numbers.txt`. Assume that the text file contains only numerical data and spaces.

```
File numData = new File("numbers.txt");
Scanner fileInput = new Scanner(numData);

double sum = 0.0;
while (fileInput.hasNext())
{
  /* missing code */
}

System.out.println(sum);
```

Which of the following can be used to replace `/* missing code */` so that the code segment prints the sum of the positive numbers in the file?

Unit 4 Progress Check: MCQ Part B

(A)

```
if (fileInput > 0)
{
  sum = sum + fileInput;
}
```

(B)

```
if (fileInput.nextDouble() > 0)
{
  sum = sum + fileInput.nextDouble();
}
```

(C) ✓

```
double temp = fileInput.nextDouble();
if (temp > 0)
{
  sum = sum + temp;
}
```

(D)

```
double temp = fileInput.nextDouble();
if (temp > 0)
{
  sum = sum + fileInput.nextDouble();
}
```

Unit 4 Progress Check: MCQ Part B

Answer C

Correct. Each iteration of the loop checks whether there is another value to be read from the file. If so, this value is obtained by the call `nextDouble()` and stored in the variable `temp`. If this value is positive, it is added `sum`.

Unit 4 Progress Check: MCQ Part B

3. A text file named data.txt has the following contents. Assume that the file contains no spaces.

```
123
456
789
```

In the following code segment, a valid Scanner object named source is created to read from the text file.

```
int x = 0;

File text = new File("data.txt");
Scanner source = new Scanner(text);

while (source.hasNext())
{
  x = source.nextInt();
}
System.out.println(x);
```

What is printed as a result of executing the code segment?

(A)

0

(B)

123

(C) ✓

789

(D)

123456789

Answer C

Unit 4 Progress Check: MCQ Part B

Correct. The code segment uses nextInt() to obtain each int value from the file. Each value obtained is assigned to the variable x. The value 789 is the last value assigned to the variable before the loop is terminated and the value is printed.

4. Which of the following statements will store the value 34.0 in the double variable value?

(A)

```
double value = (double) "34";
```

(B) ✓

```
double value = Double.parseDouble("34");
```

(C)

```
double value = Integer.parseInt("34") + ".0";
```

(D)

```
double value = Double.parseDouble("34") + ".0";
```

Answer B

Correct. The method parseDouble takes a string argument and returns the argument as a double value.

Unit 4 Progress Check: MCQ Part B

5. Consider the following calculate method.

```
public double calculate(double x)
{
  return x + 1.5;
}
```

The following code segment calls the method calculate in the same class.

```
Double d1 = new Double(7.5);
System.out.println(calculate(d1));
```

What, if anything, is printed when the code segment is executed?

(A)

8.5

(B)

9

(C) ✓

9.0

(D) Nothing is printed because the code does not compile. The actual parameter d1 passed to calculate is a Double, but the formal parameter x is a double.

Answer C

Correct. The variable d1, a Double with the value 7.5, is passed to the calculate method. Since the formal parameter of calculate is a double, Java automatically unboxes the actual parameter. The variable x is incremented by 1.5, resulting in 9.0, and that value is returned and printed.

Unit 4 Progress Check: MCQ Part B

6. A text file named grades.txt has the following contents.

89 92 90 88 79 85 77

In the following code segment, a valid Scanner object named scan is created to read from the text file. The code segment is intended to store the values from grades.txt into an int array.

```
File gradeFile = new File("grades.txt");
Scanner scan = new Scanner(gradeFile);

int[] reportCard = new int[7];
for (int i = 0; i < reportCard.length; i++)
{
  int currentGrade = /* missing expression */ ;
  reportCard[i] = currentGrade;
}
```

Which of the following can replace /* missing expression */ so that the code segment works as intended?

(A)

Double.parseDouble(scan.next())

(B) ✓

Integer.parseInt(scan.next())

(C)

Integer.parseInt(scan.nextInt())

(D)

Integer.parseInt(scan.nextLine())

Answer B

Correct. Each call to scan.next() returns a String from the text file. For example, the first call returns "89". These String values are passed to the parseInt method, which returns an Integer value that can be unboxed into an int.

Unit 4 Progress Check: MCQ Part B

7. Consider the following statement, which is intended to create an ArrayList named al to store only elements of type Thing. Assume that the Thing class has been properly defined and includes a no-parameter constructor.

```
ArrayList<Thing> al = /* missing code */
```

Which of the following can be used to replace /* missing code */ so that the statement works as intended?

(A)

```
new Thing();
```

(B) ✓

```
new ArrayList<Thing>();
```

(C)

```
new ArrayList(Thing);
```

(D)

```
new ArrayList<>(Thing);
```

Answer B

Correct. This statement correctly declares an ArrayList that can contain only Thing objects. The reference type Thing is contained in angle brackets to indicate that the a object is to contain elements of type Thing.

Unit 4 Progress Check: MCQ Part B

8. Consider the following code segment.

```
ArrayList<Integer> ages = new ArrayList<Integer>();
ages.add(18);
ages.add(20);
ages.add(1, 15);
```

What are the contents of ages after executing the code segment?

(A)

[15, 18, 20]

(B) ✓

[18, 15, 20]

(C)

[18, 20, 15]

(D)

[20, 15, 18]

Answer B

Correct. The values 18 and 20 are added to the end of ages while 15 is added in between those values at index 1.

Unit 4 Progress Check: MCQ Part B

9. Consider the following code segment.

```
ArrayList<Integer> nums = new ArrayList<Integer>();

nums.add(3);
nums.add(2);
nums.add(1);
nums.add(0);
nums.add(0, 4);
nums.set(3, 2);
nums.remove(3);
nums.add(2, 0);
```

Which of the following represents the contents of nums after the code segment is executed?

(A)

[2, 3, 2, 1, 0]

(B)

[4, 0, 0, 1, 3]

(C)

[4, 2, 0, 2, 0]

(D) ✓

[4, 3, 0, 2, 0]

Answer D

Correct. After the call nums.add(0), the contents of nums are [3, 2, 1, 0]. The call nums.add(0, 4) inserts 4 at index 0, changing the contents to [4, 3, 2, 1, 0]. The call nums.set(3, 2) sets the element at index 3 to 2, changing the contents to [4, 3, 2, 2, 0]. The call nums.remove(3) removes the element at index 3, changing the contents to [4, 3, 2, 0]. Finally, the call nums.add(2, 0) inserts 0 at index 2, changing the contents to [4, 3, 0, 2, 0].

Unit 4 Progress Check: MCQ Part B

10. Consider the following method.

```
public static void processNumbers(ArrayList<Integer> numbers)
{
   int last = numbers.size() - 1;
   numbers.add(0, numbers.get(last));
   numbers.remove(last + 1);
}
```

Which of the following best describes the preconditions necessary for the method to run without error?

(A) The `ArrayList` must be sorted from smallest to largest.

(B) The `ArrayList` must contain at least one element. ✓

(C) The `ArrayList` must contain at least two elements.

(D) The `ArrayList` must contain only positive `Integer` values.

Answer B

Correct. At least one element is needed so that numbers.size() - 1 gives a valid index.

Unit 4 Progress Check: MCQ Part B

11. Consider the following code segment.

```
ArrayList<Integer> nums = new ArrayList<Integer>();

nums.add(12);
nums.add(13);
nums.add(15);
nums.add(10);

if (nums.get(0) > nums.get(nums.size() - 1))
{
  int x = nums.get(0);
  nums.set(0, nums.get(nums.size() - 1));
  nums.set(nums.size() - 1, x);
}
```

Which of the following best describes the behavior of the code segment?

(A) It compares all values in nums and places nums in order from smallest to largest.

(B) It compares all values in nums and places nums in order from largest to smallest.

(C) It compares the first and last elements in nums and swaps them if the first element is larger than the last. ✓

(D) It compares the first and last elements in nums and swaps them if the first element is smaller than the last.

Answer C

Correct. The code segment checks the element at index 0 and compares that to the element at size of nums minus 1, which is the last element of nums. If the element at index 0 is greater, then it swaps the elements.

Unit 4 Progress Check: MCQ Part B

12. In the following code segment, `numbers` is a properly declared `ArrayList` variable that has been initialized with the following values.

```
[13, 2, -4, 0, 34, -12, -13, 18, -27]
```

```
for (int i = 0; i < numbers.size(); i++) // Line 1
{
  if (numbers.get(i) < 0)
  {
    numbers.remove(i);
  }                    // Line 6
  i--;                 // Line 7
}
```

The code segment is intended to remove all negative values from `numbers`. However, the code segment is not working as intended.

Which of the following changes can be made so that the code segment works as intended?

(A) In line 1, changing `i < numbers.size()` to `i < numbers.size() - 1`

(B) In line 7, changing `i--;` to `i++;`

(C) Removing line 7

(D) Switching lines 6 and 7 ✓

Answer D

Correct. When a negative number is removed, the remaining elements shift to the left. The value of `i` should be decreased only after an element is removed, so `i--;` should be in the body of the `if` statement.

Unit 4 Progress Check: MCQ Part B

13. In the following code segment, intList is properly declared and initialized as an ArrayList<Integer> variable.

```
int someVal = 0;

for (int i = 0; i < intList.size(); i += 2)
{
  someVal++;
}

System.out.println(someVal);
```

Which of the following best describes the value printed as a result of executing the code segment?

(A) The sum of all the elements in intList

(B) The number of elements in intList

(C) The number of even values in intList

(D) The number of elements in even-numbered indexes in intList ✓

Answer D

Correct. The loop starts at index 0 and increments by 2 each time through the loop until it reaches the size of the list. Even though it does not access any of the elements, it effectively counts the number of even values that would be valid indexes for the list.

Unit 4 Progress Check: MCQ Part B

14. Consider the following code segment.

```
ArrayList<Integer> myList = new ArrayList<Integer>();
myList.add(75);
myList.add(20);
myList.add(99);

ArrayList<Integer> yourList = new ArrayList<Integer>();
/* missing code */
```

After the code segment is executed, yourList should contain [20, 99, 75].
Which of the following can replace /* missing code */ to produce this result?

Unit 4 Progress Check: MCQ Part B

(A) ✓

```
int initialSize = myList.size();

for (int i = 0; i < initialSize; i++)

{

  myList.add(2, myList.remove(0));

  yourList.add(myList.get(0));

}
```

(B)

```
int initialSize = myList.size();

for (int i = 1; i <= initialSize; i++)

{

  yourList.add(myList.get(i));

}
```

(C)

```
for (Integer temp : myList)

{

  yourList.add(0, temp);

}
```

Unit 4 Progress Check: MCQ Part B

(D)

```
for (Integer temp : myList)
{
  myList.add(2, myList.remove(0));
  yourList.add(temp);
}
```

Answer A

Correct. In each iteration, the first statement removes the first element of myList and adds it to the end. The second statement gets the first element of myList and adds it to the end of yourList. When the loop has completed execution, yourList has the values indicated.

Unit 4 Progress Check: MCQ Part B

15. In the following code segment, numbers is a properly declared and initialized ArrayList<Integer> variable.

```
for (int i = 0; i < numbers.size(); i++)
{
  if (numbers.get(i) < 0)
  {
    numbers.remove(i);
  }
}
```

The code segment is intended to remove all negative values from numbers. Which of the following best describes the conditions in which the code segment works as intended?

(A) The code segment works as intended for all possible contents of numbers.

(B) The code segment works as intended only when negative numbers are not in adjacent positions in numbers. ✓

(C) The code segment works as intended only when numbers is sorted.

(D) The code segment works as intended only when there are no duplicate values in numbers.

Answer B

Correct. When an element is removed from an ArrayList, the remaining elements are shifted to the left by one. This could cause elements to be skipped during a traversal. When an ArrayList has adjacent elements with negative values, the first negative element is removed and the second one is skipped, so the ArrayList will still contain elements with a negative value.

Unit 4 Progress Check: MCQ Part B

16. Consider the following method.

```
public void removeAll(int target, ArrayList<Integer> list)
{
  for (int index = list.size(); index >= 0; index--)  // Line 3
  {
    if (list.get(index) == target)                    // Line 5
    {
      list.remove(index);
    }
  }
}
```

The method is intended to remove all values matching a target value from its `ArrayList` parameter. However, it is not working as intended.

Which of the following changes can be made so that the method works as intended?

(A) In line 3, changing the loop initialization to `index = list.size() - 1` ✓

(B) In line 3, changing the loop condition to `index > 0`

(C) In line 3, changing the loop increment to `index++`

(D) In line 5, changing the list access expression to `list.get(target)`

Answer A

Correct. The code will throw an `IndexOutOfBoundsException` the first time through the loop because the index `list.size()` is beyond the last element in the `ArrayList`.

Unit 4 Progress Check: MCQ Part B

17. The following code segment initializes and processes an `ArrayList of String` elements. The loop is intended to delete every other element in the `ArrayList` starting with the first element.

```
ArrayList<String> nameList = new ArrayList<String>();
nameList.add("Stacey");
nameList.add("Bob");
nameList.add("Susan");
nameList.add("Pete");

for (int i = 0; i < nameList.size(); i++)
{
  /* missing code */
}
```

After the code segment is executed, `nameList` should contain `["Bob", "Pete"]`.

Which of the following can replace `/* missing code */` so that the code segment works as intended?

(A) ✓

```
nameList.remove(i);
```

(B)

```
nameList.remove(i);

i++;
```

(C)

```
nameList.set(i, "Bob");

nameList.set(i, "Pete");
```

(D)

```
nameList.set(i, "Bob");

nameList.set(i + 1, "Pete");
```

Answer A

Unit 4 Progress Check: MCQ Part B

Correct. In the first iteration (`i = 0`), the element at position 0 is removed and the remaining elements are moved up one position, reducing the size of the list to 3. In the second iteration (`i = 1`), the element at position 1 (originally at position 2) is removed and the remaining elements are moved up a position. At this point, the size of the list is 2, so at the next iteration (`i = 2`), the loop ending condition is `false` and the loop exits.

Unit 4 Progress Check: MCQ Part B

18. Consider the following method.

```
public static void transform(ArrayList<Integer> items)
{
  for (int i = items.size() - 1; i >= 0; i--)
  {
    items.add(i, items.get(i));
  }
}
```

The following code segment appears in a method in the same class as `transform`.

```
ArrayList<Integer> numbers = new ArrayList<Integer>();
numbers.add(15);
numbers.add(4);
numbers.add(19);
numbers.add(7);

transform(numbers);
```

What are the contents of `numbers` after executing the code segment?

(A)

[15, 4, 4, 19, 19, 7, 7]

(B)

[15, 4, 19, 7, 7, 19, 4, 15]

(C)

[15, 4, 19, 7, 15, 4, 19, 7]

(D) ✓

[15, 15, 4, 4, 19, 19, 7, 7]

Answer D

Unit 4 Progress Check: MCQ Part B

Correct. The loop traverses each element in the `ArrayList` in reverse. In each iteration, the loop extracts the current element and adds it before itself. So, for the first iteration, `7` is extracted and added between `19` and `7`, yielding `[15, 4, 19, 7, 7]`. In the second iteration, `19` is extracted and added before itself, yielding `[15, 4, 19, 19, 7, 7]`. The remaining iterations are similar so that each element is copied and inserted before the original.

Unit 4 Progress Check: MCQ Part B

19. Consider the following code segment.

```
double x = 0.0;
for (double number : numbers)
{
   x += number;
}
System.out.println(x / numbers.size());
```

Assume that numbers is an ArrayList that has been initialized with the following Double objects.

[10.0, 20.0, 30.0, 40.0, 100.0]

What is printed as a result of executing the code segment?

(A)

20.0

(B)

30.0

(C) ✓

40.0

(D)

200.0

Answer C

Correct. The code segment computes the average of the elements in numbers. It uses an enhanced for loop to compute the sum of the elements and then divides the sum by the size of the ArrayList.

Unit 4 Progress Check: MCQ Part B

20. In the following code segment, words is an ArrayList of two or more String objects.

```
int prev = words.get(0).length();
int count = 0;

for (int j = 1; j < words.size(); j++)
{
  int curr = words.get(j).length();
  if (curr == prev)
  {
    count += 1;
  }
  prev = curr;
}
```

Which of the following best describes the value assigned to count as a result of executing the code segment?

(A) The number of elements in words whose length is the same length as the first element

(B) The number of elements in words whose length is the same length as the element immediately preceding it ✓

(C) The number of elements in words whose value is equal to the first element

(D) The number of elements in words whose value is equal to the element immediately preceding it

Answer B

Correct. The code segment starts with prev equal to the length of the first element and compares that to curr, the length of the next element. It adds one to count each time prev and curr are the same length. It then updates prev to be equal to curr and updates curr to be equal to the next word, continuing until the end of the ArrayList.

Unit 4 Progress Check: MCQ Part B

21. In the code segment shown, `myList` is an `ArrayList` of integers. The code segment is intended to remove all elements with the value `0` from `myList`.

```
int j = 0;
while (j < myList.size())
{
  if (myList.get(j) == 0)
  {
    myList.remove(j);
  }
  j++;
}
```

The code segment does not always work as intended.

For which of the following lists does the code segment **not** produce the correct result?

(A)

[0, 1, 0, 2]

(B) ✓

[1, 0, 0, 2]

(C)

[1, 2, 3, 0]

(D)

[1, 2, 3, 4]

Answer B

Correct. For this `ArrayList`, the code segment removes the element at index `1`, shifting the second instance of `0` to index `1`. The code segment then increases `j` to `2`, so the second instance of `0` is never removed.